Docker Storage

Practical Assignment Report | Commands, execution steps, verification and observations

1. Aim

To understand Docker storage and practise persisting container data using Docker volumes, bind mounts, and temporary filesystem mounts.

2. Requirements

• Docker Engine or Docker Desktop installed and running.

• A terminal: Ubuntu/WSL, Linux terminal, macOS Terminal, or PowerShell with Docker CLI.

• Internet access to download the sample Nginx image, if it is not already available.

3. Docker storage concepts

Volumes

A volume is managed by Docker and is the recommended way to persist data generated by containers. A volume exists independently of a container, so its data can remain after the container is removed.

Bind mounts

A bind mount maps a specific file or directory from the host machine into a container. It is useful when a developer wants to edit files on the host and see the changes inside the container.

tmpfs mounts

A tmpfs mount stores data in host memory rather than in a persistent disk-backed Docker volume. It is temporary and is removed when the container stops.

4. Practical 1 — Create and use a Docker volume

Run these commands in your terminal:

```
docker volume create app-data

docker volume ls

docker run -d --name storage-demo -v app-data:/usr/share/nginx/html nginx
```

This creates a Docker-managed volume named **app-data** and mounts it at Nginx's document root inside the container.

5. Write data into the volume

```
docker exec storage-demo sh -c 'echo "Hello from Docker volume" > /usr/share/nginx/html/index.html'

docker exec storage-demo cat /usr/share/nginx/html/index.html
```

Expected content displayed by the final command: **Hello from Docker volume**.

6. Verify persistence

```
docker rm -f storage-demo

docker run -d --name storage-check -p 8080:80 -v app-data:/usr/share/nginx/html nginx
```

```
docker exec storage-check cat /usr/share/nginx/html/index.html
```

The same text should still be present because the named volume was not deleted. Open **http://localhost:8080** in a browser to view the page.

7. Practical 2 — Use a bind mount

Create a folder and an HTML file on the host. In Ubuntu/WSL or Linux:

```
mkdir -p ~/docker-bind-demo
printf '<h1>Hello from a bind mount</h1>\n' > ~/docker-bind-demo/index.html

docker run -d --name bind-demo -p 8081:80 \
  --mount type=bind,source="$HOME/docker-bind-demo",target=/usr/share/nginx/html \
  nginx
```

Open **http://localhost:8081**. Edit the host file and refresh the page; the change should appear because the host directory is mounted into the container.

8. Practical 3 — Use a tmpfs mount (Linux containers)

```
docker run -d --name tmpfs-demo --tmpfs /app-tmp nginx

docker exec tmpfs-demo sh -c 'echo temporary-data > /app-tmp/test.txt'
docker exec tmpfs-demo cat /app-tmp/test.txt
```

The file is available while the container is running. The tmpfs data is temporary and is not intended for persistent storage.

9. Useful inspection and cleanup commands

```
docker inspect storage-check

docker volume inspect app-data

docker ps -a

docker rm -f storage-check bind-demo tmpfs-demo
# Delete the volume only when you no longer need its data:
docker volume rm app-data
```

10. Observations / Results

• A named volume stores data independently of the container lifecycle.

• A bind mount exposes a host directory or file inside a container.

• A tmpfs mount is temporary and memory-backed.

> Submission evidence: take screenshots of the successful volume creation, data written and read back, persistence check after container recreation, bind-mount page in the browser, and tmpfs test. The expected outputs above are examples; include your own terminal screenshots as proof of execution.

11. Conclusion

Docker provides multiple storage options for different needs. Volumes are suitable for persistent container data, bind mounts are useful for sharing host files with containers, and tmpfs is appropriate for temporary data.